Приведите формальные модели шифров.

Излагайте материал системно. Для моделей шифров укажите их компоненты и назначение. Для принципов поточных шифров — поясните, для чего нужен каждый принцип (стойкость, эффективность и т.д.).

Ответ

В современной криптографии под **формальной моделью шифра** понимается математическое описание пространства возможных данных и набора алгоритмов (функций), обеспечивающих преобразование информации для защиты её конфиденциальности.
Основные формальные модели:

1. Классическая теоретико-множественная модель (Модель Шеннона)
2. Вероятностная модель (Случайные величины)
3. Формальная модель **симметричного** и **асимметричного** шифрования.

1. Классическая теоретико-множественная модель (Модель Шеннона)

Классическая математическая модель шифра. Чаще всего используется для описания базовой структуры симметричных криптосистем, где один и тот же ключ используется для зашифрования и расшифрования.
Определяется как кортеж из пяти элементов:

$$\mathcal{S} = (\mathcal{M}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$$

Компоненты и их назначение:

1. $\mathcal{M}$ **(Message Space / Пространство открытых текстов)**. Множество всех возможных осмысленных исходных сообщений (открытых текстов), которые могут быть переданы.
2. $\mathcal{C}$ **(Ciphertext Space / Пространство шифротекстов)**. Множество всех возможных зашифрованных сообщений (криптограмм), получаемых в результате применения

алгоритма шифрования.

3. $\mathcal{K}$ **(Key Space / Пространство ключей)**. Множество всех допустимых ключей, используемых для настройки алгоритмов шифрования и расшифрования.
4. $\mathcal{E} = \{E_k : k \in \mathcal{K}\}$ **(Семейство функций шифрования)**. Набор обратимых отображений вида $E_k : \mathcal{M} \to \mathcal{C}$. Функция E_k преобразует открытый текст $m \in \mathcal{M}$ в шифротекст $c \in \mathcal{C}$ с использованием конкретного ключа k.
5. $\mathcal{D} = \{D_k : k \in \mathcal{K}\}$ **(Семейство функций расшифрования)**. Набор отображений вида $D_k : \mathcal{C} \to \mathcal{M}$. Функция D_k восстанавливает исходный открытый текст m из шифротекста c.

Важным свойством модели является т.н. “Условие корректности (обратимости)”. Для любого открытого текста $m \in \mathcal{M}$ и любого ключа $k \in \mathcal{K}$ должно выполняться:

$$D_k(E_k(m)) = m$$

Главная идея модели Шеннона - **длина ключа была равна или больше длины самого сообщения, а сам ключ использовался только один раз (как в одноразовых блокнотах / шифре Вернама).**
Но в реальной практике и разработке современных шифров (таких как AES, ChaCha20 или протоколы TLS/VPN) классическая модель Шеннона в своем чистом виде не применяется т.к. идея о том. что **длина ключа была равна или больше длины самого сообщения** приведет к тому. что для шифрования файла в 10000 строк потребуется клюк в 10000 строк. Тем не менее, ее принципы используются на практике. Например, практические поточные шифры, тот же **ChaCha20**, используемый в Google и TLS 1.3 — это прямая попытка сымитировать идеальную модель Шеннона (шифр Вернама), но обойти ограничение на длину ключа:

- Вместо истинно случайного ключа огромной длины берется короткий секретный ключ (например, **256 бит**) и открытая метка времени/счетчик (Initialization Vector / Nonce).
- Алгоритм генерирует из них длинную псевдослучайную последовательность битов (**гамму**).

Эта гамма накладывается на текст через операцию **XOR**, точно так же, как в модели Шеннона. Безопасность такой схемы зависит от того, насколько сложно отличить сгенерированную гамму от истинно случайного шума.

2. Вероятностная модель (Случайные величины)

В отличие от чисто алгебраического взгляда (где шифр — это просто набор функций), вероятностная модель рассматривает открытые тексты, ключи и шифротексты как **случайные величины**, распределенные на соответствующих пространствах.

Необходимость этой модели продиктована тем, что криптостойкость оценивается через **неопределенность (энтропию)** для противника. Противник пытается угадать сообщение или ключ, имея неполную информацию, а его успех измеряется вероятностью.
Для анализа стойкости шифра (например, совершенной секретности по Шеннону) элементы модели рассматриваются как случайные величины:

- Сообщение X распределено на множестве P с вероятностью

$$P(X = x)$$

- Ключ K выбирается случайно и независимо от сообщения, обычно равновероятно:

$$Y = e_K(X)$$

Важным выводом из этой можели является то, что **шифр обладает совершенной секретностью, если перехват шифртекста не дает никакой новой информации об открытом тексте (т.н. “Условие совершенной секретности”)**. Математически это означает, что апостериорная вероятность совпадает с априорной:

$$P(X = x \mid Y = y) = P(X = x)$$

В реальной инженерной практике **вероятностная формальная модель шифра** (базирующаяся на случайных величинах, распределениях и энтропии) используется не для написания кода, а для **оценки безопасности алгоритмов, проектирования криптографических чипов и генерации ключей**.

На практике модель используется для:

1. Проектирования и аудита аппаратных генераторов случайных чисел (TRNG).

Разработчики используют вероятностную модель и тесты **NIST (SP 800-22)** или **Dieharder**, чтобы доказать, что распределение вероятностей битов на выходе генератора максимально близко к **равномерному** (как того требует теория). Если вероятность появления единицы составляет хотя бы 51% вместо 50%, вероятностная модель покажет падение энтропии, и такой генератор будет отбракован, так как он облегчает задачу взломщику.

2. Оценка стойкости к атакам по сторонним каналам (Side-Channel Attacks)

Довольно часто шифры взламывают не математически, а физически — измеряя потребление энергии процессором, электромагнитное излучение или время выполнения операции во время шифрования (например, на смарт-картах или SIM-картах). Защита от таких атак строится на методе **маскирования (Masking)**, который полностью опирается на вероятностную модель. Секретный ключ внутри чипа математически разделяется на случайные под-величины (доли) так, чтобы в каждый момент времени процессор оперировал только случайным шумом:

$$K = k_1 \oplus k_2 \oplus \cdots \oplus k_n$$

При этом распределение вероятностей энергопотребления чипа становится независимым от реального секретного ключа (K). Злоумышленник, замеряя осциллографом токи в микросхеме, видит лишь случайные распределения величин и не может извлечь ключ.

3. Формальная модель симметричного и ассиметричного шифрования

В симметричных криптосистемах один и тот же секретный ключ используется и для зашифрования, и для расшифрования. Модель формализуется тройкой вероятностных полиномиальных алгоритмов:

$$\Pi_{sym} = (\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$$

Компоненты и их назначение:

- $\mathrm{Gen}(1^n) \to k$ **(Алгоритм генерации ключа)**. Вероятностный алгоритм, который на основе параметра безопасности n (длины ключа) генерирует случайный ключ $k \in \mathcal{K}$.
- $\mathrm{Enc}(k, m) \to c$ **или** $E_k(m)$ **(Алгоритм зашифрования)**. Принимает секретный ключ k и сообщение $m \in \mathcal{M}$, возвращает шифротекст $c \in \mathcal{C}$ (может быть детерминированным или вероятностным).
- $\mathrm{Dec}(k, c) \to m$ **или** $D_k(c)$ **(Алгоритм расшифрования)**. Детерминированный алгоритм, который принимает секретный ключ k и шифротекст c, возвращая исходное сообщение m либо символ ошибки $\perp$, если шифротекст поврежден.
-

Как и в случае с моделью Шеннона, реальное применение модели требует надстроек. К примеру,в формальной модели симметричный ключ (K) должен быть случайным. Но генерировать чисто случайный ключ - очень накладная задча. Поэтому на практике для каждого нового сообщения или пакета к ключу примешивается **вектор инициализации (IV)** или **однократное число (Nonce)**:

- Оно передается открыто вместе с шифртекстом.
- Его главная задача — гарантировать, что даже если вы дважды отправляете по сети сообщение «Привет», шифртексты будут абсолютно разными.

В асимметричных системах используется пара ключей: открытый (публичный) для шифрования и закрытый (секретный) для расшифрования. Модель представляется тройкой алгоритмов:

$$\Pi_{asym} = (\mathrm{KeyGen}, \mathrm{Enc}, \mathrm{Dec})$$

Компоненты и их назначение:

1. $\mathrm{KeyGen}(1^n) \to (pk, sk)$ **(Алгоритм генерации пары ключей):**
   - *Назначение:* Вероятностный алгоритм, генерирующий пару ключей:
     - $pk \in \mathcal{PK}$ (*public key*) — открытый ключ, публикуемый в общем доступе.
     - $sk \in \mathcal{SK}$ (*secret key*) — закрытый ключ, хранящийся в секрете у получателя.
2. $\mathrm{Enc}(pk, m) \to c$ **(Алгоритм шифрования):**
   - *Назначение:* Алгоритм, позволяющий *любому* участнику сети зашифровать открытый текст $m \in \mathcal{M}$ с использованием известного открытого ключа получателя pk.
3. $\mathrm{Dec}(sk, c) \to m$ **(Алгоритм расшифрования):**
   - *Назначение:* Алгоритм, позволяющий *только законному владельцу* закрытого ключа sk восстановить сообщение m из шифротекста c.

- **Условие корректности:**

$$\forall (pk, sk) \leftarrow \mathrm{KeyGen}(1^n), \forall m \in \mathcal{M} : \mathrm{Dec}(sk, \mathrm{Enc}(pk, m)) = m$$

Как и в формальной модели симметричного шифрования, модель ассиметричного шифрвоания на практике не йиспользуется. Применяется т.н. **гибридное шифрование**.В этом случае формальная модель используется для **шифрования не самих данных, а симметричного ключа**:

1. $\mathrm{KeyGen}(1^n)$ **в действии:** Сервер (например, `google.com`) один раз генерирует пару ключей. Открытый ключ зашивается в SSL-сертификат и отдается всему миру. Секретный ключ хранится на сервере.
2. **Шаг 1 (Симметричный):** Ваш браузер заходит на сайт и генерирует случайный, короткий и очень быстрый **симметричный ключ сессии** (например, для AES-256).
3. $\mathrm{Enc}(pk, m)$ **в действии:** Браузер берет открытый ключ сервера (pk) и зашифровывает им этот *симметричный ключ* (x). Получается маленький шифртекст c.
4. **Передача:** Браузер отправляет (c) на сервер. Перехватывать его бесполезно.
5. $\mathrm{Dec}(sk, c)$ **в действии:** Сервер принимает (c), расшифровывает его своим секретным ключом ((sk)) и достает симметричный ключ сессии ((x)).
6. **Результат:** Теперь и у браузера, и у сервера есть общий быстрый ключ. Весь дальнейший массивный трафик шифруется уже по симметричной модели (AES).

На практике алгоритм зашифрования $\mathrm{Enc}(pk, m)$ **всегда вероятностный** благодаря специальным стандартам дополнения, таким как **OAEP** (Optimal Asymmetric Encryption Padding) для RSA:

- Перед шифрованием к вашему тексту (x) принудительно подмешивается случайная битовая строка (соль) и хэш-функции.
- Каждый раз при шифровании одного и того же слова «Да» на выходе получается абсолютно новый, случайный шифртекст.
- При расшифровании алгоритм автоматически отсекает эту случайную соль и выдает чистый текст.